



Word Level Timestamp Generation for Automatic Speech Recognition and Translation

Ke Hu¹, Krishna Puvvada¹, Elena Rastorgueva¹, Zhehuai Chen¹, He Huang¹, Shuoyang Ding¹,
Kunal Dhawan¹, Hainan Xu¹, Jagadeesh Balam¹, Boris Ginsburg¹

¹NVIDIA, USA

kevinhu@nvidia.com

Abstract

We introduce a data-driven approach for enabling word-level timestamp prediction in the Canary model. Accurate timestamp information is crucial for a variety of downstream tasks such as speech content retrieval and timed subtitles. While traditional hybrid systems and end-to-end (E2E) models may employ external modules for timestamp prediction, our approach eliminates the need for separate alignment mechanisms. By leveraging the NeMo Forced Aligner (NFA) as a teacher model, we generate word-level timestamps and train the Canary model to predict timestamps directly. We introduce a new `<|timestamp|>` token, enabling the Canary model to predict start and end timestamps for each word. Our method demonstrates precision and recall rates between 80% and 90%, with timestamp prediction errors ranging from 20 to 120 ms across four languages, with minimal WER degradation. Additionally, we extend our system to automatic speech translation (AST) tasks, achieving timestamp prediction errors around 200 milliseconds. Our code is open-sourced through NeMo¹, and the checkpoint is released at Hugging Face².

Index Terms: Word timestamp, ASR, AST

1. Introduction

Accurate timing information is highly desirable for automatic speech recognition (ASR) and downstream tasks such as speech content retrieval (e.g., for lookup of speech content by keywords) and generating timed subtitles. In addition, recent works show that transcription with word-level timestamps also facilitates other speech tasks such as speech-to-speech conversation [1] or decoding with time flow information [2].

There have been a number of works developing timestamp predictions for various ASR models. Traditional hybrid systems [3], which are trained with phoneme alignments, naturally provide alignments between speech frames and text. In the E2E framework, although models like connectionist temporal classification (CTC) are not directly trained with phoneme or word alignments, one can still use Viterbi decoding to the log-probabilities output by CTC models and reference text for forced alignment (e.g., [4]). Other E2E models perform timestamp prediction as a separate pass in addition to regular decoding. Systems like WhisperX [5] run voice activity detection and a separate forced phoneme alignment to generate word-level timestamps. WhisperTimestamped and related methods [6, 7] first generate cross attention weights between speech and predicted words during decoding and then use dynamic time warping to generate a word-level alignment. These methods usually need a long chunk of speech to be available for aligning.

Instead of using a separate module or method to generate timestamps, there have been works on enabling E2E models to directly emit timestamps. For example, forced alignment generated by a conventional ASR model is used to teach RNN Transducer (RNN-T) to emit timestamps [8] or guide the RNN-T training to emit word tokens at the specified time [9]. Recently, [10] proposes a time-token transducer to jointly model label and token duration prediction. While the model is efficient in decoding, it may delay the duration emission which leads to inaccurate word-level start and end timestamps. Whisper [11] and OWSM [12] use a data-driven approach to directly train the model to emit timestamps. However, the timestamps are at the *segment level* due to training data limitation. Recently, OpenAI released the feature to generate *word-level* timestamps for Whisper [13], but it is unclear how the feature is developed.

In this work, we research a data driven approach to enable word-level timestamp prediction for the Canary model [14]. To overcome the training data scarcity issue, we use the NeMo Forced Aligner (NFA) [4] as the teacher to generate word-level timestamps for training our timestamp model. We note that the proposed method can work with any teacher data at either word or segment level. To add the timestamp prediction ability, we introduce a new `<|timestamp|>` prompt for the original Canary model as a new task. Similar to [11, 12], we model timestamps as special tokens up to a maximum duration (i.e., 36 sec in this work). We show that our approach can achieve decent precision and recall (ranging from 80% to 90%) for timestamp prediction and the prediction errors ranges from the 20 to 120 ms range for 4 languages, with slight degradation on ASR quality. In addition to ASR, we further extend the system to perform word-level timestamp prediction for automatic speech translation (AST). We define this task as, for each translated word, predicting the start and end frame positions of that word in the original speech. We also follow a data driven approach for this new task. To generate teacher time alignments, we first use NFA to align source speech and source text at the word level, and then use awesome-align [15] to obtain alignment between source and target text at the word level. The results show that our model can predict word-level timestamps with average errors in the range of 200 ms for 6 language pairs, but this comes with a translation performance drop of 4.4 BLEU points or 2.6 COMET score.

The novelty of our work is two-fold: 1) We propose a solution for multilingual word-level timestamp prediction using teacher generated data. The ASR timestamp prediction has a good coverage of precision and recall at around 90%, and prediction errors in the range of 50-60 ms on average, and 2) we show that by a data-driven teacher-student learning, the model is also capable of predicting word-level timestamps for speech translation. As far as we know, our work represents the first investigation for word-level timestamp prediction for AST.

¹<https://github.com/NVIDIA/NeMo>

²<https://huggingface.co/nvidia/canary-1b-flash>

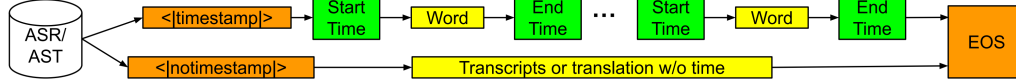


Figure 1: *Timestamp training data format.* Our training data consists of both ASR or AST training data. Either `<|timestamp|>` and `<|notimestamp|>` prompt tokens are used for prompting. Both start and end timestamps are added for each word in training.

2. Modeling Details

We use the Canary model [14] as the baseline for adding timestamp ability. Canary is a multilingual model which is capable of multiple tasks such as ASR and AST for English, French, Spanish, and German. Canary models multitasking by using various prompts tokens, such as `<|transcribe|>`, `<|translate|>`, `<|pnc|>`, `<|nopnc|>`. To add timestamp prediction ability to Canary, we introduce `<|timestamp|>` and `<|notimestamp|>` for generating word-level timestamps or no timestamps, respectively.

As shown in Fig. 1, our training data is formatted in two styles, `<|timestamp|>` or `<|notimestamp|>`. In the `<|timestamp|>` case, we add start and end time tokens for each word in the sentence. The timestamps are first obtained by force aligning the speech and ground truth text using NFA [4], and then we take the start and end time (in sec) and convert them to frame indices using 80 ms frame rate (determined by the NFA encoder frame rate). Similar to [11], we represent the timestamp tokens as `<|t|>`, where `t` is an integer ranging from 0 to 450, representing a maximum duration of 36 sec. For example, our transcripts with timestamps look like: `<|3|> classifying <|14|> <|15|> was <|16|> <|18|> everything <|19|> <|23|> to <|24|> <|25|> him <|26|>`. Each timestamp token is tokenized to exactly one ID in the tokenizer output space. To tokenize transcripts, similar to [14], we use SentencePiece [16] and concatenated tokenizer [17] with a vocabulary size of 1024 for each supported language and a sub-tokenizer for the special tokens. Using special tokens enables a one-to-one mapping between the timestamp and the output ID, which helps in reducing the target sequence length. We have also tried to tokenize the timestamps directly using the SentencePiece tokenizer but did not get reasonable performance. Our model is multilingual and we use the same special tokens for timestamps across different languages. We also tried predicting the time shift from the previous timestamp to current timestamp (similar to duration) but did not manage to get good performance.

In AST training, our AST text translations are generated synthetically using machine translation models from ASR transcripts [14]. Therefore, we have both ASR transcripts and AST translations. To generate timestamps for AST, we first align speech with ASR transcripts using NFA, and then use awesome-align [15] to align ASR transcripts (source) with AST translations (target) at the word level. Timestamps of each source word are then copied to the corresponding target word. Since awesome-align [15] shows reasonable zero-shot performance, we use it for aligning between English and three languages: French, German, and Spanish, even though Spanish has not been seen in their training data. We have obtained reasonable performance as shown in the results.

3. Experiments

3.1. Data and Evaluation Metrics

We use a subset of Canary training data [14] to generate timestamps, i.e., 4,275, 1,410, 1,397 and 1,795 hours of speech with word-level timestamps for English, German, Spanish and French, respectively. We use the NeMo Forced Aligner (NFA) [4] to align the training data for each language separately. To prevent regression, we also use non-timestamped data [14] in training. Overall, the timestamp data accounts for 15% of all ASR training data. We use equal weights between ASR and AST timestamp training data. We evaluate model performance for ASR timestamp prediction using LibriSpeech test-other [18] and MCV-16.1 [19]. For AST, we use FLEURS [20] to evaluate our timestamp prediction performance for 6 language pairs, i.e. $En \rightarrow X$ and $X \rightarrow En$, where X is German, French or Spanish. ASR performance is measured using WER, and we use BLEU [21] and COMET [22] for AST evaluation.

For timestamp evaluation, we use two types of metrics: 1) Precision and recall, and 2) start-time difference (SD) and end-time difference (ED). For precision & recall, as in [4], we compute the true positive (TP) as number of correctly predicted timestamped word in the hypothesis, false positive (FP) as the incorrectly predicted timestamped words in the hypothesis, and false negative (FN) as the number of timestamped words in the reference but missing or incorrectly predicted in the hypothesis. The precision and recall are then calculated as $precision = \frac{TP}{(TP+FP)}$ and $recall = \frac{TP}{(TP+FN)}$. We define a “timestamped word” as a predicted word with estimated start and end timestamps, and we consider a timestamped word to be correct if: 1) Exact match of the words between a hypothesis and reference, and 2) Both start and end timestamps differ by less than 240 ms. The chosen threshold of 240 ms (instead of 200 ms in [4]) is because our encoder has an encoding rate of 80 ms.

We further use start and end time differences (SD & ED) to quantify the accuracy of the predicted timestamps. Here, we only calculate the SD and ED differences when there is a word match between the hypothesis and reference. The absolute differences are calculated no matter whether the difference is less or more than the 240 ms threshold.

3.2. Training Details

We use a 170M Canary L model [14] in our experiments. The model consists of a 17-layer conformer encoder, with a model dimension of 512 and projection dimension of 2048. The decoder is a regular 17-layer transformer decoder [23] with 512-dimensional layers. A downsampling factor of 8 is used to speed up decoding. We use fixed positional embedding for the decoder. In training, we initialize from an existing checkpoint of a Canary L model [14] and continue to fine tune the whole model. Due to the increased timestamp tokens, we do not initialize the softmax and the embedding layers from the checkpoint. Our input features are 128-dim log-mel features with 10-ms frame rate. Lhotse [24] is used for dataloading with a

Metric	En		Es	Fr	De	Avg.
	LS test-other	MCV-16.1	MCV-16.1	MCV-16.1	MCV-16.1	
Precision (%)	93.7	83.0	93.5	85.4	90.7	89.3
Recall (%)	93.6	84.5	93.9	88.1	91.1	90.2
SD (ms)	50	69	22	22	112	55
ED (ms)	54	70	42	36	127	66

Table 1: *Timestamp prediction results in precision & recall, and start & end time differences w.r.t. the NFA ground truth.*

Model	En		Es	Fr	De	Avg.
	LS test-other	MCV-16.1	MCV-16.1	MCV-16.1	MCV-16.1	
Canary Baseline (B0)	4.8	11.7	5.7	15.0	7.0	8.8
+ start & end TS data (E1)	5.0	12.1	5.9	14.8	7.1	9.0

Table 2: *WER comparison between the baseline Canary model and the proposed model.*

Error Threshold	Precision / Recall (%)	
	E1	WhisperTimestamped
240 (ms)	95.8 / 95.6	83.6 / 83.2
320 (ms)	96.1 / 95.9	92.8 / 92.2
400 (ms)	96.3 / 96.1	95.3 / 94.7
480 (ms)	96.4 / 96.2	96.1 / 95.5

Table 3: *Comparison of the proposed model and WhisperTimestamped [6] based on the LibriSpeech test-other set.*

batch duration of 600 sec per GPU. For model training, we use AdamW optimizer and inverse square root annealing learning rate scheduler with a learning rate of 1e-3 and a warm-up step of 2.5k. We implement the model with PyTorch using the NeMo Toolkit [25], and the model is trained on 32 A100 (80G) GPUs for 30k steps, with a batch duration of 600 sec per GPU. In training, we simply sum over the loss for word and timestamp tokens without any priors in weighting.

4. Results

4.1. Timestamp Prediction for ASR

4.1.1. Timestamp Prediction Quality

As shown in Table 1, we evaluate the timestamp prediction performance of the proposed method. We use precision and recall to measure the overall timestamp prediction accuracy. The NFA alignments generated by ground truth ASR transcripts are used as the references. As we can see from Table 1, our model achieved a precision of 93.7% for LibriSpeech test-other and 89.3% on average. The recalls are 93.6% and 90.2%, respectively. This shows that our model can accurately capture most words with their timestamps. We then calculate the predicted SD and ED for each word when there is a word match. As shown in Table 1, the SD and ED range from 22 to 127 ms. In particular, the predictions are fairly accurate for English, Spanish, and French, and relatively worse for German.

4.1.2. WER Regression

In addition to timestamp prediction performance, we also want to make sure the model does not degrade performance on ASR transcripts. Therefore, in Table 2, we compare the WER of the baseline Canary model (B0) and the proposed model with timestamp data in training (E1). The baseline Canary model is trained without any timestamp data, and fine tuned for the same number of training steps as the timestamp model for fair comparison. To calculate the WER For E1, we use the `<|timestamp|>` to prompt the model but ignore the timestamp prediction errors and only compute the WER of the words. Table 2 shows that there is only small degradation for ASR

performance (around 0.2% on average) compared to baseline. Additionally, we have also tried prompting the model with `<|notimestamp|>` to only generate ASR transcripts without timestamps and there is no WER regression at all.

4.1.3. Comparison

We have compared to WhisperTimestamped [6] in Table 3 using the LibriSpeech test-other set. For WhisperTimestamped, we use the “small” model, which has a similar size (244M) to our model (175M). We note that WhisperTimestamped [6] derives timestamps in an unsupervised way by aligning each word to a contiguous number of speech frames using dynamic time warping (DTW) based on cross-attention weights, and thus have a single timestamp between two words. Therefore, we use that timestamp as the start timestamp of current word as well as the end timestamp of the previous word. Due to the formatting, the derived start and end timestamps from WhisperTimestamped may inherently differ from the NFA produced “ground truth”, and this should not be penalized in evaluation. We therefore do not calculate the absolute differences between the predicted and reference start and end timestamps but present precision and recall percentages. To minimize differences in ground truth labeling, we also vary the match threshold from 240 to 480 ms to calculate precision and recall for both models. When the threshold is 240 ms, we see that our model performs significantly better than WhisperTimestamped. As we increase the threshold (i.e., relaxing the ground truth labeling convention), Table 3 shows that our model (E1) consistently outperforms WhisperTimestamped across all thresholds.

4.2. Timestamp Prediction for AST

We use FLEURS [20, 14] to evaluate the timestamp prediction performance for AST. We initially follow the steps described in Sect. 2 to generate per-word timestamps for evaluation. However, due to the lack of strict word-to-word mapping caused by translation reordering and errors, it is challenging to directly compare word-level timestamps between ground truth and hypotheses. In addition, the timestamp estimation should not be penalized by incorrect text translations. Therefore, instead of using the ground truth translation for the awesome-align we use the hypotheses (removing timestamps) as the target for alignment. The aligned results are used as references in evaluation. In this way, we have one-to-one correspondence for each word, and the start and end timestamps between references and hypotheses.

As shown in Table 4, the task of predicting timestamps for AST is in general more difficult than ASR. The average start and end time errors are 204 and 224 ms, respectively, across

Metric	Timestamp Prediction Errors						
	De→En	Fr→En	Es→En	En→De	En→Fr	En→Es	Avg
SD (ms)	323	94	173	228	223	181	204
ED (ms)	364	115	199	232	236	198	224

Table 4: Timestamp prediction performance for speech translation using the FLEURS test set.

Model	BLEU / COMET						
	De→En	Fr→En	Es→En	En→De	En→Fr	En→Es	Avg
B0	29.0 / 80.9	28.3 / 81.1	20.1 / 79.6	25.2 / 74.2	34.5 / 76.6	19.6 / 77.1	26.1 / 78.3
E1	23.4 / 78.4	22.9 / 79.2	16.3 / 77.7	20.3 / 71.3	30.6 / 73.3	16.5 / 74.3	21.7 / 75.7

Table 5: Translation performance regression in BLEU and COMET using the FLEURS test set.

Predicted German Hypothesis	English Ground Truth
< 0 > Jedoch < 21 > < 21 > aufgrund25 < 28 > der < 29 > < 29 > langsamen < 33 > < 34 > Kommunikationskanäle < 43 > < 43 > könnten < 55 > < 55 > Stile < 62 > < 64 > im < 65 > < 67 > Westen < 71 > < 91 > um < 92 > < 95 > fünfundzwanzig < 96 > < 104 > bis < 105 > < 106 > dreißig < 110 > < 111 > Jahre < 112 > < 79 > hinterherbleiben < 83 >	< 0 > However, < 20 > < 23 > due < 26 > < 26 > to < 28 > < 28 > the < 29 > < 30 > slow < 33 > < 34 > communication < 42 > < 43 > channels, < 52 > < 55 > styles < 62 > < 64 > in < 65 > < 65 > the < 67 > < 67 > west < 71 > < 73 > could < 74 > < 76 > lag < 79 > < 81 > behind < 82 > < 88 > by < 89 > < 92 > 25 < 97 > < 100 > to < 101 > < 103 > 30 < 106 > < 108 > year < 121 >

Table 6: The proposed model is capable of learning the syntactic reordering in speech translation.

Predicted English Hypothesis	French Ground Truth
< 0 > The < 1 > < 22 > surface < 28 > < 28 > of < 29 > < 29 > the < 30 > < 31 > moon < 34 > < 35 > is < 36 > < 36 > constituted < 42 > < 42 > of < 43 > < 44 > stone < 47 > < 48 > and < 49 > < 49 > of < 50 > < 50 > duster < 56 > < 58 > its < 60 > < 60 > louch < 64 > < 65 > exterior < 71 > < 72 > is < 73 > < 74 > called < 77 > < 78 > the < 79 > < 80 > croute < 87 >	< 0 > La < 1 > < 22 > surface < 28 > < 28 > de < 29 > < 29 > la < 30 > < 31 > lune < 34 > < 35 > est < 36 > < 36 > constituée < 42 > < 42 > de < 43 > < 43 > pierres < 48 > < 48 > et < 49 > < 49 > de < 50 > < 50 > poussière. < 58 > < 58 > Sa < 59 > < 60 > couche < 64 > < 65 > extérieure < 71 > < 73 > est < 74 > < 74 > appelée < 78 > < 78 > la < 79 > < 80 > croûte. < 88 >

Table 7: The proposed model shows capability of predicting timestamps for speech translation when trained only on ASR timestamp data.

6 En→X and X→En pairs. Among different language pairs, the performance is significantly worse between German and English. This is probably because the different grammatical structure of German (such as syntactic reordering), which makes timestamp prediction more challenging.

In addition, we evaluate the AST quality regression by BLEU and COMET. We use a <|timestamp|> prompt to generate the text with timestamps for E1 and then remove timestamps to only evaluate the text translation. We use SacreBLEU [21] for BLEU calculation and the Unbabel/wmt22-comet-da model from HuggingFace [26] for COMET [27] score calculation. We normalize the COMET score to [0, 100] by multiplying the raw score by 100. As shown in Table 5, the translation ability of the timestamp model degrades by 4.4 BLEU points, while the degradation with respect to the COMET score is 2.6 points.

4.3. Example and Discussion

In this section, we show some interesting examples of the predicted sentences with timestamps. Table 6 shows an example of English speech to German text translation. Note that our model predicts the German verb with timestamps (i.e., <|79|> hinterherbleiben <|83|>) at the end of the sentence, with corresponding start and end timestamps in the English speech (i.e., <|76|> lag <|79|> <|81|> behind <|82|>). This re-ordering of hinterherhinken is due to the subject-object-verb (SOV) structure of German, whereas the model still preserves the timestamps of the original English speech. We note that the timestamps predicted here are no longer monotonically increasing but follow the correspondence of the original English speech. We also note that here two words in English is mapped

to one word in German. It would be interesting and worth discussing how the model should handle non-consecutive mapping between words.

Another interesting observation from our training is that a timestamp model trained using only ASR timestamp data demonstrates the capability of predicting timestamps for AST. We present a French-to-English AST example in Table 7. The predicted timestamps appear quite reasonable when compared with the French ground truth in the second column of Table 7. Again, we note that the model has not been trained on any AST training data with timestamps. Although the timestamps seem reasonable, this model suffers from substantial translation quality regression from 26.1 to 9.7 BLEU points on average. Some common translation errors (bold in the first column) appear to be incorrectly predicting English words to be their French counterparts with similar pronunciation. Future research may focus on few-shot learning for AST timestamp prediction.

5. Conclusion

We propose a data-driven approach for word-level timestamp prediction for a Canary ASR and AST, leveraging NeMo Forced Aligner (NFA) and awesome-align as teacher models. Our method achieves 80-90% precision and recall with timestamp errors of 20-120 ms for ASR. Extending to AST, we observed 200 ms timestamp errors with a 4.4 BLEU and 2.6 COMET drop. Our model performs better than WhisperTimestamped in ASR timestamp prediction. We will release the code and checkpoints to support open-source development. Future work includes refining AST timestamping to reduce translation degradation.

6. References

- [1] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, and N. Zeghidour, “Moshi: a speech-text foundation model for real-time dialogue,” *arXiv preprint arXiv:2410.00037*, 2024.
- [2] F. Seide, M. Doulaty, Y. Shi, Y. Gaur, J. Jia, and C. Wu, “Speech reallm–real-time streaming speech recognition with multimodal llms by teaching the flow of time,” *arXiv preprint arXiv:2406.09569*, 2024.
- [3] M. McAuliffe, M. Socolof, S. Mihuc, M. Wagner, and M. Sonderegger, “Montreal forced aligner: Trainable text-speech alignment using kaldi,” in *Interspeech*, vol. 2017, 2017, pp. 498–502.
- [4] E. Rastorgueva, V. Lavrukhin, and B. Ginsburg, “Nemo forced aligner and its application to word alignment for subtitle generation,” in *Proc. INTERSPEECH*, 2023.
- [5] M. Bain, J. Huh, T. Han, and A. Zisserman, “Whisperx: Time-accurate speech transcription of long-form audio,” *arXiv preprint arXiv:2303.00747*, 2023.
- [6] J. Louradour, “whisper-timestamped,” <https://github.com/linto-ai/whisper-timestamped>, 2023.
- [7] L. Wagner, B. Thallinger, and M. Zusag, “Crisperwhisper: Accurate timestamps on verbatim speech transcriptions,” *arXiv preprint arXiv:2408.16589*, 2024.
- [8] “Emitting word timings with end-to-end models,” in *Interspeech*. IEEE, 2020.
- [9] R. Zhao, J. Xue, J. Li, W. Wei, L. He, and Y. Gong, “On addressing practical challenges for rnn-transducer,” in *2021 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. IEEE, 2021, pp. 526–533.
- [10] H. Xu, F. Jia, S. Majumdar, H. Huang, S. Watanabe, and B. Ginsburg, “Efficient sequence transduction by jointly predicting tokens and durations,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 462–38 484.
- [11] A. Radford, J. W. Kim, T. Xu *et al.*, “Robust speech recognition via large-scale weak supervision,” in *ICML*. PMLR, 2023, pp. 28 492–28 518.
- [12] Y. Peng, J. Tian, B. Yan, D. Berrebbi, X. Chang, X. Li, J. Shi, S. Arora, W. Chen, R. Sharma *et al.*, “Reproducing whisper-style training using an open-source toolkit and publicly available data,” in *2023 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. IEEE, 2023, pp. 1–8.
- [13] OpenAI, “Speech-to-text: Timestamps guide,” 2024, accessed: 2024-10-01. [Online]. Available: <https://platform.openai.com/docs/guides/speech-to-text/timestamps>
- [14] K. C. Puvvada, P. Želasko, H. Huang, O. Hrinchuk, N. R. Koluguri, K. Dhawan, S. Majumdar, E. Rastorgueva, Z. Chen, V. Lavrukhin *et al.*, “Less is more: Accurate speech recognition & translation without web-scale data,” *arXiv preprint arXiv:2406.19674*, 2024.
- [15] Z.-Y. Dou and G. Neubig, “Word alignment by fine-tuning embeddings on parallel corpora,” *arXiv preprint arXiv:2101.08231*, 2021.
- [16] T. Kudo, “Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing,” *arXiv preprint arXiv:1808.06226*, 2018.
- [17] K. Dhawan, K. Rekesh, and B. Ginsburg, “Unified model for code-switching speech recognition and language identification based on concatenated tokenizer,” in *Proceedings of the 6th Workshop on Computational Approaches to Linguistic Code-Switching*, 2023, pp. 74–82.
- [18] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “Librispeech: an asr corpus based on public domain audio books,” in *2015 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2015, pp. 5206–5210.
- [19] R. Ardila, M. Branson, K. Davis, M. Henretty, M. Kohler, J. Meyer, R. Morais, L. Saunders, F. M. Tyers, and G. Weber, “Common voice: A massively-multilingual speech corpus,” *arXiv preprint arXiv:1912.06670*, 2019.
- [20] A. Conneau, M. Ma, S. Khanuja *et al.*, “Fleurs: Few-shot learning evaluation of universal representations of speech,” in *SLT*. IEEE, 2023, pp. 798–805.
- [21] M. Post, “A call for clarity in reporting bleu scores,” *arXiv preprint arXiv:1804.08771*, 2018.
- [22] R. Rei, C. Stewart, A. C. Farinha, and A. Lavie, “Comet: A neural framework for mt evaluation,” *arXiv preprint arXiv:2009.09025*, 2020.
- [23] A. Vaswani, “Attention is all you need,” *Advances in Neural Information Processing Systems*, 2017.
- [24] P. Želasko, D. Povey, J. Trmal, S. Khudanpur *et al.*, “Lhotse: a speech data representation library for the modern deep learning ecosystem,” *arXiv preprint arXiv:2110.12561*, 2021.
- [25] O. Kuchaiev, J. Li, H. Nguyen *et al.*, “Nemo: a toolkit for building ai applications using neural modules,” *arXiv preprint arXiv:1909.09577*, 2019.
- [26] Unbabel, “wmt22-comet-da,” <https://huggingface.co/Unbabel/wmt22-comet-da>, n.d., accessed: 2024-09-25.
- [27] R. Rei, J. G. De Souza, D. Alves, C. Zerva, A. C. Farinha, T. Glushkova, A. Lavie, L. Coheur, and A. F. Martins, “Comet-22: Unbabel-ist 2022 submission for the metrics shared task,” in *Proceedings of the Seventh Conference on Machine Translation (WMT)*, 2022, pp. 578–585.